



CMPT 295

Unit - Instruction Set Architecture

Lecture 26 – ISA Example: MIPS

Last Lecture

- 3 common Types of **Instruction Set Architecture**
 1. *Stack*
 2. *Accumulator*
 3. *General Purpose Register (GPR)*
- Types of **Instruction Set**: **CISC** and **RISC**
- Instruction set (**IS**) design guidelines

Today's Menu

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design guidelines
 - Example of an instruction set: MIPS
 - Create our own instruction sets
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Pipelined execution of machine instructions + Hazards



Microprocessor without
interlocked
pipelined
stages

Example of another ISA: **MIPS**

Example of an ISA: **MIPS**

1. Memory model

➤ **Word size** -> 32 bits

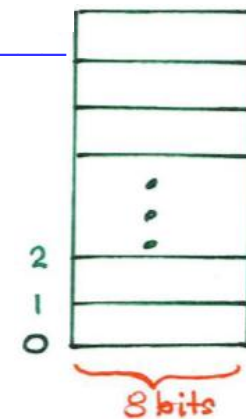
➤ **Size of memory** -> $2^m \times n$ bits

➤ Size of a memory address (# of bits) is m bits = _____ bits

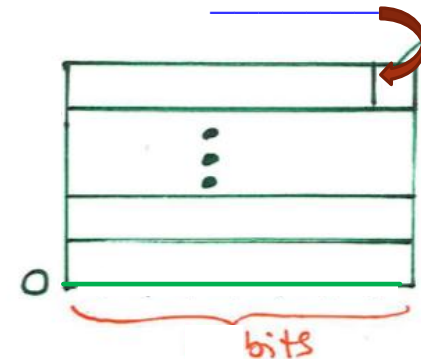
➤ Byte-addressable memory so address resolution is n bits = _____

➤ Size of memory -> _____

➤ So, there are _____ distinct addressable memory locations and each of these addressable memory locations has _____ bits.



* uncompressed view of memory.



* compressed view of memory.

MIPS Memory Model

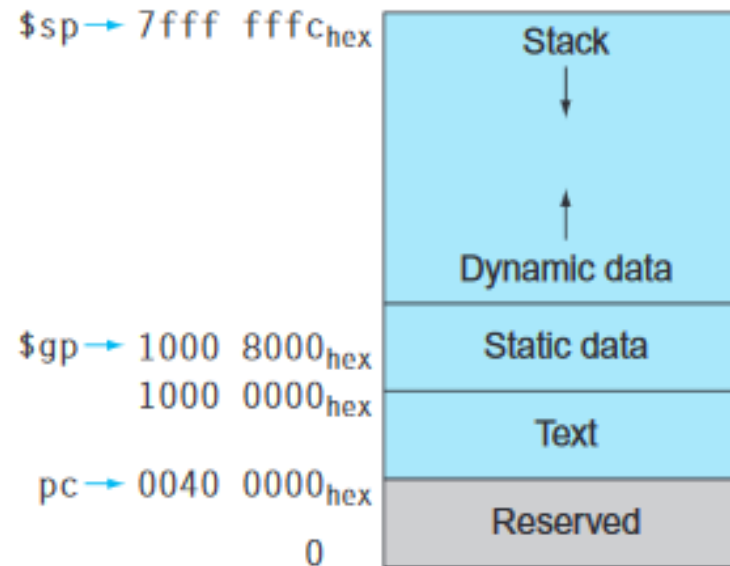


FIGURE 2.13 The MIPS memory allocation for program and data. These addresses are only a software convention, and not part of the MIPS architecture. The stack pointer is initialized to 7fff fffc_{hex} and grows down toward the data segment. At the other end, the program code (“text”) starts at 0040 0000_{hex}. The static data starts at 1000 0000_{hex}. Dynamic data, allocated by `malloc` in C and by `new` in Java, is next. It grows up toward the stack in an area called the heap. The global pointer, `$gp`, is set to an address to make it easy to access data. It is initialized to 1000 8000_{hex} so that it can access from 1000 0000_{hex} to 1000 ffff_{hex} using the positive and negative 16-bit offsets from `$gp`. This information is also found in Column 4 of the MIPS Reference Data Card at the front of this book.

2. Registers

- **# of registers:** 32 registers
- **Size of register:** 32 bits
- **Data type:**
- **Purpose of each register:**
See the **Usage** column in this Figure A.6.1

Register name	Number	Usage
\$zero	0	constant 0
\$at	1	reserved for assembler
\$v0	2	expression evaluation and results of a function
\$v1	3	expression evaluation and results of a function
\$a0	4	argument 1
\$a1	5	argument 2
\$a2	6	argument 3
\$a3	7	argument 4
\$t0	8	temporary (not preserved across call)
\$t1	9	temporary (not preserved across call)
\$t2	10	temporary (not preserved across call)
\$t3	11	temporary (not preserved across call)
\$t4	12	temporary (not preserved across call)
\$t5	13	temporary (not preserved across call)
\$t6	14	temporary (not preserved across call)
\$t7	15	temporary (not preserved across call)
\$s0	16	saved temporary (preserved across call)
\$s1	17	saved temporary (preserved across call)
\$s2	18	saved temporary (preserved across call)
\$s3	19	saved temporary (preserved across call)
\$s4	20	saved temporary (preserved across call)
\$s5	21	saved temporary (preserved across call)
\$s6	22	saved temporary (preserved across call)
\$s7	23	saved temporary (preserved across call)
\$t8	24	temporary (not preserved across call)
\$t9	25	temporary (not preserved across call)
\$k0	26	reserved for OS kernel
\$k1	27	reserved for OS kernel
\$gp	28	pointer to global area
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	return address (used by function call)

FIGURE A.6.1 MIPS registers and usage convention.

Source: Page A-24 in Patterson and Hennessy

Example of an ISA: **MIPS**

3. Instruction set

➤ **Operand model:** 3 operand assembly language

- Requiring all instructions to have 3 operands keeps all instructions the same length and this keeps the design of the microprocessor hardware simple
- Hardware for a variable number of operands is more complicated

➤ **Description (semantic):** **MIPS** assembly language notation

add a, b, c **Meaning: $a = b + c$**

What an instruction means -> **semantic**
How the instruction is formed (how we write the instruction) -> **syntax**

Activity

- If we want to write an assembly program in MIPS that sums four variables **b**, **c**, **d** and **e**, how many **add** instructions would we need?
- Answer:

Example of an ISA: **MIPS**

➤ (Sub)set of instructions

MIPS assembly language instructions	Semantic (i.e., Meaning)	Corresponding MIPS machine instructions
lw \$s1, 20(\$s2)	$\$s1 = M[\$s2 + 20]$	?
sw \$s1, 20(\$s2)	$M[\$s2 + 20] = \$s1$	?
add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	Example: 0x00af8020
sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	?
beq \$s1, \$s2, 25	if ($\$s1 == \$s2$)	?
j 2500		?
jal 2500		?

- **Memory addressing modes** -> **Direct (absolute)** and **base + displacement**
- **Format** of corresponding **MIPS** machine code instructions?
 - Length of machine code instruction -> 32 bits (**4 bytes**)
 - Size of opcode? Size of other fields? Order of operands?

↑
Format of these bit patterns?

A closer look at **MIPS**' **add** instruction

- **MIPS** assembly language instruction:

add \$s0, \$a1, \$t7

- Corresponding **MIPS** machine code instruction:

0x00af8020

Corresponding
machine
code instruction

Format of
corresponding
machine
code instruction

0000	0000	1010	1111	1000	0000	0010	0000
000000	00101	01111	10000	00000	100000		
opcode	rs	rt	rd	shamt	func		

A closer look at the **fields** of this format

Format of
R-type MIPS
machine code
instructions

opcode	rs	rt	rd	shamt	func
--------	----	----	----	-------	------

- **opcode**: operation of the instruction
- **rs**: first register source operand
- **rt**: second register source operand
- **rd**: register destination operand (contains results of operation)
- **shamt**: shift amount
- **func**: function – often called *function code*, which indicates the specific variant of the operation in the **opcode** field

➤ 6 bits

➤ 5 bits

➤ 5 bits

➤ 5 bits

➤ 5 bits

➤ 6 bits

Total: 32 bits

Does **MIPS** follow the **ISA** design guidelines?

1. Each **MIPS** instruction has an unambiguous **binary encoding**
2. **MIPS** is functionally complete, i.e., “Turing complete”
3. In terms of **machine code instruction format**:

- **Can all MIPS machine code instructions have the same format?**

- For example: `lw $s1, 20($s2)` =>

opcode	rs	rt	rd	shamt	func
--------	----	----	----	-------	------

- When designing its corresponding machine code instruction ...

- Can the **source register** be specified using 5 bits? -> Yes!
- Can the **destination register** be specified using 5 bits? -> Yes!
- Can a constant be specified using 5 bits? -> Hum...
 - Value of constant limited to $[0..2^5-1]$
 - Often this constant is used to access array elements so it needs to be $> 2^5 = 32$



MIPS ISA designers compromise to the format principle

3. In terms of **machine code instruction format**:
- a. **Ideally**, the machine code instructions in this **IS** must all have the **same length** and use the **same format**!
 - b. If we cannot achieve **a.**, we must create as few formats as possible
 - c. If we cannot achieve **a.**, then let's try to position the **fields** of our **machine code instruction formats** that have the same **purpose** at the **same location** in the **machine code instruction formats**

a. All machine code instructions the **same length** -> **32 bits**, but **do not use the same format**

b. 3 different formats for **MIPS** machine code instructions

➤ **R-format** for register

opcode	rs	rt	rd	shamt	func
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

➤ **I-format** for immediate

opcode	rs	rt	Address/immediate
6 bits	5 bits	5 bits	16 bits

➤ **J-format** for jump

opcode	Target address
6 bits	26 bits

➤ **opcode** indicates the instruction as well as the format of the instruction

➤ This way, the hardware knows whether to treat the last half of the instruction as 3 fields (**R-format**) or as 1 field (**I-format**)

c. Since we have different machine code instruction formats, **fields** with **same purpose** are positioned in the **same location** in the 3 formats ☺

Summary

- Looked at an example of a RISC instruction set: MIPS
- MIPS follows ISA design guidelines
 1. Unambiguous **binary encoding** of machine code instruction
 2. MIPS functionally complete (“Turing complete”)
 3. Only 3 machine code instruction **formats**, but all have the same length
 - *R-format* for register
 - *I-format* for immediate
 - *J-format* for jump
 - Also, position of **fields** with **same purpose** are in the **same location** in all 3 formats

Next lecture

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design guidelines
 - Example of an instruction set: MIPS
 - Create our own instruction sets
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Pipelined execution of machine instructions + Hazards